

# PSR-PIPELINE: A Software Research Framework for sEMG Finger-Intent Decoding in Post-Stroke Neurorehabilitation

PSR Project Team

July 28, 2026

## Abstract

Post-stroke finger-intent decoding from surface electromyography (sEMG) is a promising control pathway for rehabilitation wearables, but progress is often difficult to interpret because datasets, preprocessing, models, and deployment assumptions are reported in isolation. This paper presents the software portion of the PSR-PIPELINE research project, whose long-term goal is to serve compact machine-learning models on Raspberry Pi 5-class hardware for a finger-assist rehabilitation system [17, 8, 18]. The current software stack ingests raw PHYSIOMIO recordings, aligns gesture labels to five-finger multilabel targets, applies a fixed 20–450 Hz Butterworth filter, Symlet-4 wavelet denoising, and 200 ms windows with 50% overlap, extracts twelve handcrafted descriptors per channel-window pair, and feeds a shared tensor representation into LSTM, graph neural network, and 1D CNN model families. Under this common pipeline, the three direct model families reveal complementary strengths rather than a single dominant baseline: the LSTM achieves the best exact-match accuracy among the original baselines (0.545), the GNN attains the highest baseline macro F1 (0.706) and macro AUPRC (0.776), and the legacy CNN trails both. Subsequent architectural refinement shifts the frontier toward compact convolutional models: the Optuna-tuned CNN-Large student reaches 0.593 exact-match accuracy, 0.798 finger accuracy, 0.714 macro F1, 0.881 macro AUROC, and 0.812 macro AUPRC on the impaired-arm test split, slightly exceeding the best committed ResNet teacher on most aggregate metrics while remaining within an embedded-oriented size envelope. Two improvement pathways are also now visible in the repository: healthy-to-impaired transfer learning clearly benefits the CNN branch but not the LSTM branch, and teacher-ensemble distillation is implemented as a speed-quality pathway even though a fully committed distilled-student evaluation bundle is not yet present. We therefore present this manuscript as an evidence-constrained research paper on a developing neurorehabilitation software stack rather than as a claim of clinical readiness or a cross-paper state-of-the-art result.

## 1 Introduction

Finger-intent prediction from surface electromyography (sEMG) is an appealing interface for post-stroke neurorehabilitation because it offers a non-invasive path from residual muscle activity to assistive control, therapy feedback, and fine-motor retraining [14, 22, 4]. In the broader PSR project, that decoding capability is intended to support machine-learning models that can eventually be served on Raspberry Pi 5-class hardware and coupled to a rehabilitation glove. This manuscript focuses on the software research layer of that agenda: the data handling, signal processing, modeling, and evaluation work that turns multichannel sEMG into five-finger intent predictions.

The challenge is not only raw predictive accuracy, but also methodological reproducibility. Post-stroke sEMG is noisy, patient-specific, and highly sensitive to recording conditions, impairment

severity, and dataset construction, so benchmark results depend heavily on how signals are filtered, denoised, segmented, labeled, tensorized, and split across patients. Reviews of sEMG-based gesture recognition consistently note that preprocessing design, feature representation, and deployment constraints can matter as much as the final classifier family [12, 15]. A research project that reports model performance without a stable preprocessing and evaluation path is therefore difficult to audit and even harder to extend toward embedded rehabilitation use.

PSR-PIPELINE addresses that gap by providing one auditable software framework for the project [17]. The current implementation includes PHYSIOMIO ingestion, gesture-to-finger label mapping, a fixed signal-conditioning stack, window-level feature extraction, shared tensor adapters, direct LSTM/CNN/GNN baselines, a merged 1D CNN student/teacher suite, Optuna-based hyperparameter optimization, healthy-to-impaired transfer-learning summaries, and benchmark artifacts stored as committed metric files. Recent project updates also added teacher-ensemble distillation, per-finger threshold tuning, and latency benchmarking utilities, which materially strengthen the edge-deployment research story even when the outputs of those utilities are not yet fully committed as benchmark bundles.

The central story of this paper is therefore research-oriented rather than repository-oriented. We do not present a new decoding architecture, nor do we claim that the current implementation already demonstrates end-to-end rehabilitation hardware validation. Instead, we report a software research program whose current contribution lies in making experimental assumptions explicit: which recordings are used, how they are preprocessed, how tensors are shared across model families, how baseline and improved architectures are compared, which metrics are computed, and which deployment-facing hooks are already present in code.

This paper makes four study-level contributions. First, it formalizes the current software pipeline from PHYSIOMIO ingestion through preprocessing, feature extraction, tensor adaptation, model fitting, and deployment-screening utilities. Second, it establishes a clear benchmark narrative that begins with three direct model families, LSTM, CNN, and GNN, under one preprocessing regime before analyzing subsequent improvements. Third, it evaluates two concrete improvement pathways now visible in the repository: compact CNN student and ResNet teacher development, and healthy-to-impaired transfer learning, while also documenting distillation as an implemented but not yet fully benchmarked speed-quality path. Fourth, it situates the project against the broader literature on post-stroke sEMG decoding, including PhysioMio, Ninapro, and recent transfer-learning studies, while drawing a careful boundary around what the current evidence still cannot support, especially distilled-student claims, on-device Raspberry Pi 5 validation, and clinical outcome conclusions.

## 2 Related Work

sEMG has long been studied as a non-invasive control channel for rehabilitation, prosthetics, and human-machine interaction, and stroke-specific reviews show that sEMG-driven interventions can support meaningful upper-limb rehabilitation when signal interpretation is reliable enough for assistive use [14, 22]. Broader reviews of wearable sensing for stroke recovery make a related point: clinical value depends on the full stack, including acquisition quality, body placement, labeling protocol, feature representation, and the way machine learning is coupled to assessment or assistive control [4]. For this reason, decoding architecture should not be discussed independently of data construction and deployment context.

Dataset choice is especially important in this area. Ninapro remains the dominant public benchmark family for hand-gesture decoding from sEMG, with ten sub-datasets spanning different electrode

layouts, movement vocabularies, and sensor modalities [3, 16]. The widely used DB5 split, for example, records 52 movements from 10 healthy participants with two Myo armbands, while DB1 and DB2 cover larger healthy cohorts with different channel configurations and richer kinematic metadata [3, 16]. By contrast, PHYSIOMIO was introduced in 2026 as a stroke-specific bilateral and longitudinal HD-sEMG resource with 48 stroke patients, 16 gestures, 64 electrodes, and repeated recordings across inpatient recovery [8]. That distinction matters because results on healthy-subject multiclass gesture sets are not automatically transferable to impaired-arm intent decoding in post-stroke populations.

Stroke-specific decoding studies have historically been small and heterogeneous. Lee et al. [10] used subject-specific myoelectric pattern classification for six functional hand movements in chronic stroke survivors and reported a sharp impairment-dependent gap, with mean accuracy of 71.3% for moderately impaired participants and 37.9% for severely impaired participants. Anastasiev et al. [2] later studied post-acute stroke gesture recognition with portable eight-channel sEMG and found that affected-side SVM performance could still reach the high-80% range on reduced gesture sets, but the task remained sensitive to feature design, label count, and paretic signal quality. These studies underscore a recurring challenge for post-stroke intent decoding: strong performance is possible, but dataset scale and protocol differences make general claims difficult.

Deep neural models broaden that picture but do not remove the comparability problem. Sequence-oriented recurrent models remain natural baselines because they aggregate temporal evidence across windows, while compact 1D CNNs emphasize local temporal motifs and can be compressed aggressively for efficient inference [7]. Graph neural networks offer a complementary structural view by representing windows as related observations connected through message passing rather than as a purely linear sequence [9, 5, 20]. On healthy-dataset benchmarks such as Ninapro, deeper end-to-end networks can reach very high multiclass accuracies; for example, Sri-Iesaranusorn et al. [19] reported 93.87% accuracy on Ninapro DB5 and 91.69% on DB7 for 41-movement classification. Those results are useful context for model capacity, but they address a different population and a different task than impaired-arm multilabel finger-intent prediction.

Transfer learning has recently become one of the most relevant directions for rehabilitation-oriented sEMG. Li et al. [11] showed that transfer-based CNNs can materially improve inter-subject and inter-day robustness on high-density healthy-subject sEMG, and Mohammadian et al. [13] demonstrated an especially important stroke-specific result: pretraining on healthy Ninapro DB5 data raised stroke-survivor gesture-classification accuracy from 46% to 93.6% in a three-gesture setting. These findings motivate the healthy-to-impaired transfer pathway in PSR-PIPELINE, but they also raise a cautionary point. Transfer-learning gains are task-, architecture-, and dataset-dependent, so a project paper should report them as empirical outcomes rather than as guaranteed improvements.

Deployment-oriented optimization provides the final piece of context for this manuscript. Knowledge distillation is a standard way to transfer behavior from larger teachers into smaller students [6], and Optuna is widely used for black-box hyperparameter search under practical constraints [1]. What distinguishes PSR-PIPELINE is therefore not a novel recurrent, convolutional, or graph formulation in isolation, but an integrated software research effort that places dataset harmonization, baseline comparison, teacher-student modeling, transfer learning, and deployment proxies inside one auditable experimental framework.

Table 1: Literature context used to position the current study. Reported numbers come from different populations, sensors, gesture vocabularies, and label spaces, so the table is for context rather than for a direct leaderboard comparison.

| Source                       | Population dataset                                                                       | / Task                                                                           | Reported performance                                                               | Relevance to this paper                                                                                      |
|------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Lee et al. 2010              | 20 chronic stroke survivors; 10 forearm/hand electrodes                                  | Six functional hand movements with subject-specific EMG classification           | Mean accuracy 71.3% for moderate impairment and 37.9% for severe impairment        | Early evidence that stroke severity strongly affects decoder quality                                         |
| Anastasiev et al. 2022       | 19 post-acute stroke patients; portable eight-channel sEMG                               | Four- to seven-gesture multiclass recognition on affected and non-affected sides | Affected-side SVM accuracy reached 88.7% on the four-gesture setting               | Shows that reduced-vocabulary stroke gesture recognition can reach high accuracy with careful feature design |
| Mohammadiazni et al. 2026    | 10 stroke patients plus healthy Ninapro DB5 pre-training data                            | Three-gesture classification across multiple arm postures with transfer learning | 93.6% with transfer learning versus 46% without transfer                           | Strong stroke-specific motivation for healthy-to-impaired transfer learning                                  |
| Sri-Iesaranusorn et al. 2021 | Ninapro DB5/DB7, mostly healthy-subject settings                                         | 41-movement deep neural network classification                                   | 93.87% on DB5 and 91.69% on DB7                                                    | Healthy-data ceiling is high, but the task is easier than impaired-arm multilabel finger decoding            |
| PSR-PIPELINE (this paper)    | PHYSIOMIO impaired-arm split from 48 stroke patients; patient-level train/val/test split | Five-finger multilabel intent decoding from processed HD-sEMG segments           | Best committed model: CNN-Large with 0.593 exact-match accuracy and 0.714 macro F1 | Stricter label space and patient split make this a harder but more rehabilitation-specific benchmark         |

## 3 Method

### 3.1 Overview

PSR-PIPELINE implements the project’s modular decoding workflow, which starts from raw multi-channel sEMG and gesture labels and ends with either five-finger logits or downstream evaluation artifacts. The experimental design is easiest to read as four connected bands: data harmonization and ingestion, signal processing and feature extraction, model fitting, and evaluation or deployment-facing analysis. Figure 1 summarizes that workflow at the paper level. The figure includes modeling and deployment-oriented blocks that are present in the implementation, but the empirical claims in this paper remain restricted to committed artifacts.

For notation, let a pre-segmented recording be  $X \in \mathbb{R}^{T \times C}$ , where  $T$  is the number of raw time samples and  $C = 64$  is the channel count under the PHYSIOMIO path. After preprocessing and windowing, the project extracts a feature tensor  $\mathcal{X} \in \mathbb{R}^{C \times W \times F}$ , where  $W$  is the number of windows and  $F = 12$  is the number of handcrafted descriptors per channel-window pair. Each sample is paired with a multilabel target vector  $y \in \{0, 1\}^5$  indicating intended activation of thumb, index, middle, ring, and little finger.

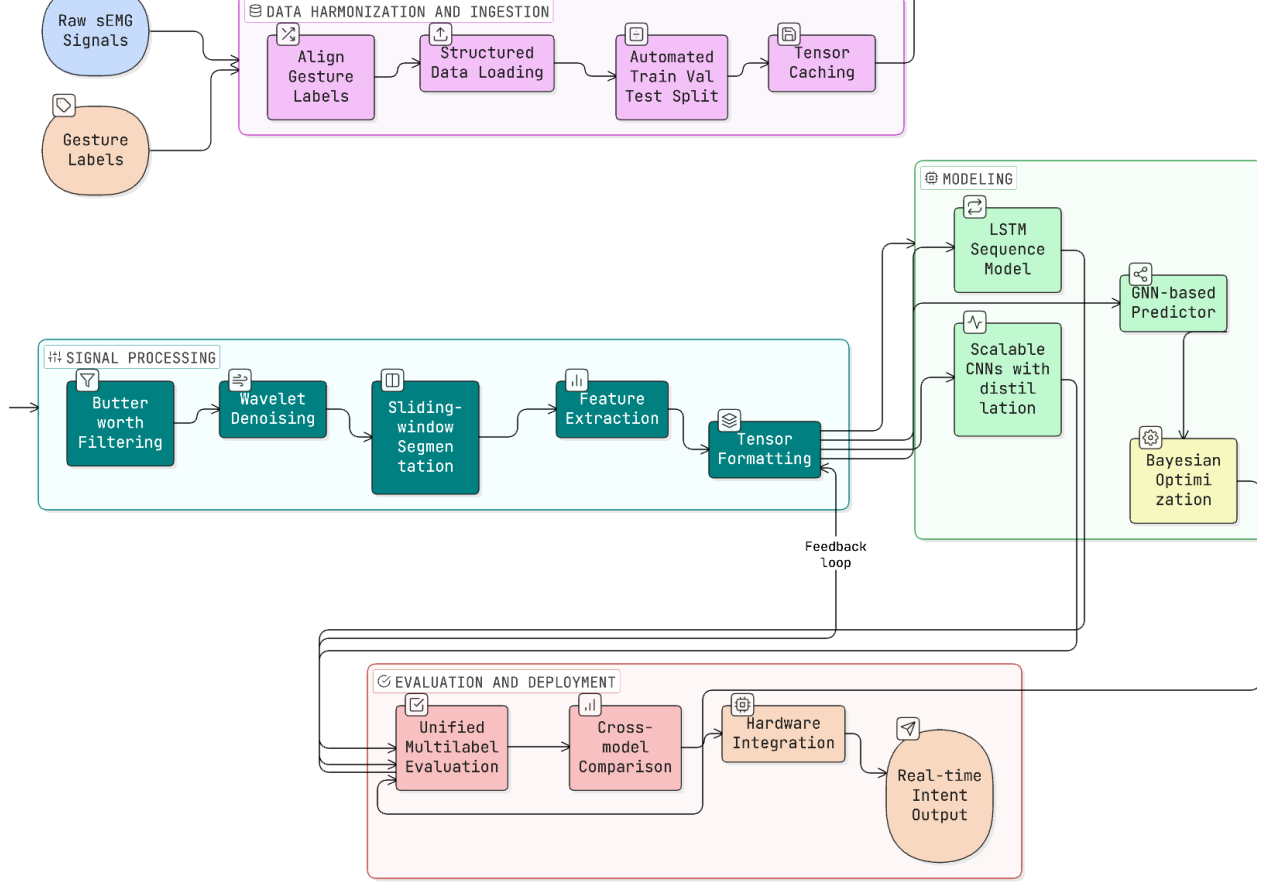


Figure 1: Paper-level view of the current project workflow. The implementation covers data harmonization, preprocessing, feature extraction, tensor formatting, multiple predictor families, unified evaluation, and deployment-facing utilities such as threshold tuning and latency benchmarking. Hardware integration and real-time output are project targets rather than experimentally validated claims in this manuscript.

### 3.2 Data harmonization and ingestion

The project loader builds processed datasets directly from raw PHYSIOMIO parquet files when cached tensors are missing. It searches the raw data tree, groups files by patient, aligns contiguous stretches of the same `movement_type`, maps each label to a five-bit finger target, discards segments shorter than 200 samples, and then applies the full preprocessing pipeline to each usable segment. These ingest operations define the effective task boundary: which arm is selected, how gestures become multilabel targets, and whether repeated data from the same patient can leak across train and test.

The default configuration uses PhysioMio recordings sampled at 2000 Hz, selects the impaired arm unless the user explicitly changes the arm split, and performs a deterministic patient-level 70/10/20 train/validation/test split. Processed samples are padded to a dataset-wide maximum window count so they can be stacked into tensors. The resulting dataset is therefore not just a collection of windows; it is a patient-split, segment-level benchmark derived from raw multichannel recordings.

### 3.3 Signal preprocessing

The preprocessing module is explicitly parameterized by a frozen configuration object in the implementation. Raw sEMG first passes through a fourth-order Butterworth band-pass filter with 20–450 Hz cutoffs and zero-phase forward-backward application. This stage suppresses low-frequency motion artifacts and high-frequency electrical noise while preserving time alignment across channels.

The filtered signal is then denoised with wavelet soft-thresholding. The default configuration uses a Symlet-4 basis, four decomposition levels, universal threshold selection, and a median-based noise estimate. Although the project does not yet include an ablation isolating this choice, the denoising rule is explicit, versioned, and reusable across all model families.

After denoising, the continuous recording is segmented into 200 ms windows with 50% overlap and no zero-padding. These settings balance temporal responsiveness against the sample count required for stable summary statistics. They are also consistent with the project’s edge-oriented framing: the models target compact downstream inference and therefore use short windows that could plausibly support reactive control.

### 3.4 Window-level feature representation

Each channel-window pair is summarized by twelve handcrafted descriptors spanning time and frequency domains. The time-domain set includes root mean square, mean absolute value, integrated EMG, waveform length, variance, zero crossings, slope sign changes, and Willison amplitude. The frequency-domain set includes mean frequency, median frequency, spectral entropy, and total power, with spectral quantities estimated from Welch periodograms using `nperseg = min(256, len(x))` [21]. Degenerate spectra are zeroed explicitly to avoid unstable feature values. The resulting tensor has shape  $(C, W, F)$ , where  $C$  is the number of channels,  $W$  is the number of windows, and  $F = 12$  is the feature count.

### 3.5 Tensor formatting and shared adapter

The project standardizes cross-model comparison with one shared adapter. It transforms  $(C, W, F)$  or  $(N, C, W, F)$  tensors into  $(N, W, C \times F)$  sequences, where windows act as time steps and all channel features are concatenated into one vector per step. For a single sample, the adapted sequence can be written as

$$S = [s_1, \dots, s_W]^\top \in \mathbb{R}^{W \times d}, \quad s_w = \text{vec}(\mathcal{X}_{:,w,:}), \quad d = CF = 768.$$

This interface decision is small but important because it lets recurrent, convolutional, and graph-based predictors share the same derived representation without duplicating preprocessing code.

For the CNN branch, the adapted tensor is subsequently permuted to a channels-first layout. The merged CNN stack treats the  $64 \times 12$  channel-feature grid as 768 input channels for temporal `Conv1d` layers, which preserves a common upstream pipeline while letting the convolutional branch operate as a lightweight sequence model over window order.

### 3.6 Predictor families

The LSTM branch applies two stacked recurrent layers with hidden sizes 128 and 256, followed by a 128-unit fully connected layer and dropout before five output logits. Given adapted sequence input

$s_t$ , the recurrent dynamics follow the standard LSTM update

$$\begin{aligned} i_t &= \sigma(W_i s_t + U_i h_{t-1} + b_i), \\ f_t &= \sigma(W_f s_t + U_f h_{t-1} + b_f), \\ o_t &= \sigma(W_o s_t + U_o h_{t-1} + b_o), \\ \tilde{c}_t &= \tanh(W_c s_t + U_c h_{t-1} + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ h_t &= o_t \odot \tanh(c_t), \end{aligned}$$

and the project uses the final hidden state from the second recurrent layer for classification. This branch emphasizes temporal accumulation across windows and serves as the main sequential baseline for the direct benchmark.

The GNN branch interprets each time window as a node in a graph. The implementation supports GCN, GraphSAGE, and GAT operators [9, 5, 20], and the benchmarked path constructs a dense graph over windows within each sample before graph-level pooling and readout. At a generic level, the node update can be written as

$$h_i^{(\ell+1)} = \psi_\ell \left( h_i^{(\ell)}, \text{AGG}_{j \in \mathcal{N}(i)} \phi_\ell(h_i^{(\ell)}, h_j^{(\ell)}) \right),$$

followed by graph pooling

$$g = \text{POOL}_{i=1}^W h_i^{(L)}$$

and a multilabel readout head. This branch injects an explicit structural prior: windows are related observations rather than only a flat sequence.

The CNN branch is the most elaborate family in the current project. It includes five student variants ranging from Nano to XLarge, each built around a cheap  $1 \times 1$  projection, temporal convolution blocks, adaptive pooling, and a small fully connected head. If the channels-first representation is denoted by  $\hat{S} \in \mathbb{R}^{d \times W}$ , the student stack can be summarized as

$$H^{(0)} = \rho(\text{BN}(W_p * \hat{S} + b_p)), \quad H^{(\ell+1)} = \rho(\text{BN}(W_\ell * H^{(\ell)} + b_\ell)),$$

where  $*$  denotes 1D convolution over window order and  $\rho$  is a ReLU nonlinearity. Adaptive pooling then compresses the temporal axis before a small multilayer perceptron produces the five logits. The accompanying README reports approximate footprints from 54K parameters and 53 KB INT8 for Nano up to 1.62M parameters and 1.58 MB INT8 for XLarge, while the teacher family includes 1D ResNet-50, ResNet-101, and ResNet-152 variants. The key methodological idea is that all of these models sit behind the same input adaptation pathway, so increasing model capacity does not require changing ingestion or preprocessing logic.

### 3.7 Training, tuning, and evaluation protocol

The training utilities unify optimization and reporting across model families. The current implementation uses binary cross-entropy with logits for five-label prediction, Adam-based optimization, plateau-triggered learning-rate reduction, early stopping, checkpointing, and saved metric curves. For logits  $z \in \mathbb{R}^5$ , the base multilabel objective is

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{5} \sum_{k=1}^5 [y_k \log \sigma(z_k) + (1 - y_k) \log (1 - \sigma(z_k))].$$



Evaluation reports exact-match accuracy, finger-level averaged accuracy, macro precision, macro recall, macro F1, macro AUROC, and macro AUPRC. The metrics module also stores per-finger scores and task-level curve data, which enables both aggregate and finger-wise analysis after training.

Hyperparameter optimization is a first-class component rather than an afterthought. The merged CNN stack uses Optuna to search architecture and training choices for each student size, and its score combines validation accuracy with a latency-based penalty relative to a baseline timing reference [1]. Trial-level early stopping is used during search so poorly performing candidates terminate quickly. The project also contains two deployment-oriented utilities: a teacher-ensemble distillation path that can fuse multiple teachers either equally or by per-finger validation performance, and a threshold-tuning script that sweeps per-finger thresholds over a validation grid to maximize F1. For multilabel distillation, the implemented loss blends the hard target term with a temperature-scaled soft target from the teacher,

$$\mathcal{L}_{\text{KD}} = \alpha \mathcal{L}_{\text{BCE}}(z_s, y) + (1 - \alpha) T^2 \text{BCELogits}\left(\frac{z_s}{T}, \sigma\left(\frac{z_t}{T}\right)\right),$$

where  $z_s$  and  $z_t$  are student and teacher logits,  $T$  is the distillation temperature, and  $\alpha$  controls the hard-soft balance. A separate latency benchmark reports mean, median, and 95th-percentile inference time on a chosen device. Together, these components extend the work beyond a pure offline benchmark and toward a practical edge-oriented study.

### 3.8 Transfer-learning path

In addition to direct impaired-arm benchmarks, the project stores two-stage summaries for CNN and LSTM models that pretrain on healthy-arm data and finetune on impaired-arm data. This procedure preserves the same task definition and objective while changing the source domain from healthy to impaired-arm recordings. These artifacts matter for post-stroke applications because impaired-arm data are often scarcer and harder to collect than healthy data. Although the project does not yet harmonize every transfer run into the same directory structure as the direct benchmark, the committed summaries are sufficient to study whether the two-stage path helps each model family.

## 4 Experiments

### 4.1 Dataset and split construction

All benchmark artifacts analyzed in this paper come from the project’s PHYSIOMIO processing path implemented in PSR-PIPELINE [8, 17]. The loader searches raw parquet files, extracts contiguous movement segments, maps gesture labels to five-bit finger targets, preprocesses each segment, and stores processed tensors as PyTorch split files.

The split logic is patient-level rather than segment-level. Patients are shuffled with a fixed seed and divided into 70% train, 10% validation, and 20% test partitions, which reduces identity leakage between train and test segments. The default loader configuration targets impaired-arm recordings unless the user explicitly changes the arm-split setting, so the main benchmark should be interpreted as an impaired-arm post-stroke decoding study. The two-stage transfer summaries use the same software pathway with healthy-arm pretraining followed by impaired-arm finetuning.



## 4.2 Evidence scope

The project exposes a richer experimental surface than a single comparison table. In addition to direct LSTM, CNN, and GNN runs, the implementation contains optimized CNN student and teacher evaluations, healthy-to-impaired pretraining and finetuning summaries, optional threshold tuning, teacher-ensemble distillation, and latency benchmarking. However, not all branches are represented by equally complete committed result bundles. This paper therefore separates three evidence tiers: direct committed test metrics, committed auxiliary summaries such as transfer learning and CNN evaluation sweeps, and code-only deployment utilities whose behavior is documented but whose outputs are not yet comprehensively committed.

The project has also evolved over time. Historical direct baselines remain under `metrics/`, while newer CNN outputs live under `models/CNN/evaluations/`. We keep both because they jointly show how the study matured from an initial cross-family comparison into a more deployment-oriented CNN branch. When a newer optimized artifact supersedes an older baseline, the paper says so explicitly rather than averaging across incompatible runs.

## 4.3 Evaluation protocol

The project defines the task as five-label multilabel classification with a nominal decision threshold of 0.5 on the held-out test split. For transparency, the metric implementation records both exact-match accuracy and finger-wise summaries, which is important because different models can trade precision for recall while achieving similar per-label correctness. The headline tables in this paper use the metrics as stored in the committed result files. Threshold-tuning utilities are described in the methods and discussion but are not used to create new manuscript numbers.

Because the benchmark mixes exact-match, per-finger, and ranking-sensitive metrics, the evaluation section is organized deliberately. We first compare the three direct model families under the shared preprocessing path. We then examine whether the newer CNN branch materially changes the frontier by comparing optimized student variants against the internal ResNet teacher family merged through PR 56. Finally, we treat transfer learning and distillation as improvement pathways rather than as baseline-defining results, because their artifact completeness differs.

This paper follows an evidence-constrained writing policy. Every headline claim is tied to a committed code path, metric file, or project artifact. Because the current evidence base does not yet include a full ablation package, an on-device Raspberry Pi 5 benchmark log, or a polished distilled-student result bundle, we deliberately avoid module-level causality claims and instead focus on benchmark-level observations that are directly supported by saved outputs.

# 5 Results

## 5.1 Direct baseline models: LSTM, CNN, and GNN

The first layer of evidence is the direct three-family comparison that motivated the project before later CNN refinements were merged. Table 2 shows the held-out impaired-arm metrics for the original LSTM, CNN, and GNN runs under the shared preprocessing path. No single family dominates every metric. The LSTM is the strongest baseline on exact-match accuracy and macro AUROC, the GNN is best on macro F1 and macro AUPRC, and the legacy CNN is competitive on finger accuracy but otherwise trails the other two branches.

Table 2: Held-out test performance for the three direct model families. Bold numbers mark the best score in each column within this baseline comparison only.

| Model      | Exact-match  | Finger Acc.  | Macro F1     | Macro AUROC  | Macro AUPRC  |
|------------|--------------|--------------|--------------|--------------|--------------|
| LSTM       | <b>0.545</b> | <b>0.784</b> | 0.705        | <b>0.858</b> | 0.754        |
| CNN legacy | 0.442        | 0.694        | 0.676        | 0.767        | 0.764        |
| GNN        | 0.448        | 0.683        | <b>0.706</b> | 0.787        | <b>0.776</b> |

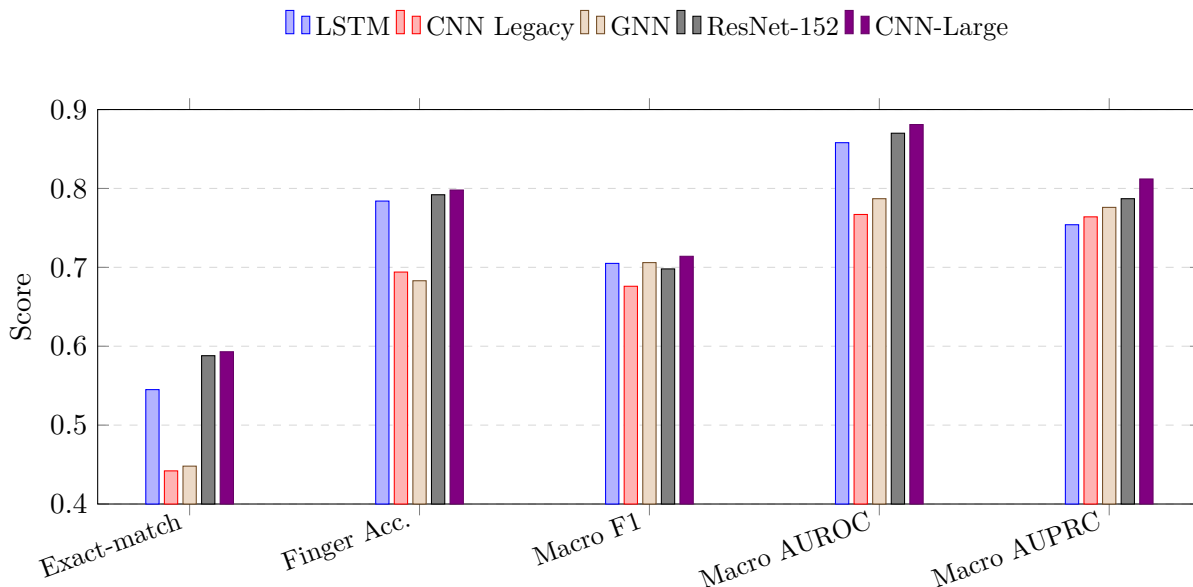


Figure 2: Aggregate-metric comparison across the three direct baselines plus the strongest internal teacher and student references from the newer CNN branch.

This baseline table is important because it supports the paper’s first claim: the project did not begin as a single-model CNN paper. It began as a cross-family study in which recurrent, graph, and convolutional views of the same feature representation each offered different trade-offs. That observation still matters after the newer CNN work is added, because it explains why the repository contains multiple model families and why finger-level analysis remains useful even when one branch improves the aggregate scores.

## 5.2 Performance-improvement path: optimized CNN students and ResNet teachers

The second layer of evidence asks whether the repository’s newer CNN branch materially shifts the benchmark frontier. Table 3 answers yes. Relative to the legacy CNN baseline, the optimized student family improves all major aggregate metrics, and the strongest student, CNN-Large, becomes the best committed model artifact in the repository with 0.593 exact-match accuracy, 0.798 finger accuracy, 0.714 macro F1, 0.881 macro AUROC, and 0.812 macro AUPRC.

Several observations follow from Table 3. First, the change from the legacy CNN to the optimized CNN-Large artifact is substantial rather than cosmetic: exact-match accuracy rises from 0.442 to 0.593, macro F1 from 0.676 to 0.714, macro AUROC from 0.767 to 0.881, and macro AUPRC

Table 3: Detailed CNN-family evaluation. Student parameter counts and INT8 sizes come from the project CNN README; teacher rows are included as internal high-capacity reference models merged through PR 56. Bold numbers mark the best score in each column across the rows shown.

| Variant    | Type    | Params | INT8 Size | Exact-match  | Finger Acc.  | Macro F1     | Macro AUROC  | Macro AUPRC  |
|------------|---------|--------|-----------|--------------|--------------|--------------|--------------|--------------|
| ResNet-50  | Teacher | –      | –         | 0.570        | 0.788        | 0.656        | 0.840        | 0.772        |
| ResNet-101 | Teacher | –      | –         | 0.569        | 0.795        | 0.681        | 0.852        | 0.767        |
| ResNet-152 | Teacher | –      | –         | 0.588        | 0.792        | 0.698        | 0.870        | 0.787        |
| Nano       | Student | 54K    | 53 KB     | 0.580        | 0.794        | 0.698        | 0.855        | 0.758        |
| Micro      | Student | 158K   | 154 KB    | 0.578        | 0.794        | 0.697        | 0.871        | 0.793        |
| Base       | Student | 390K   | 381 KB    | 0.575        | <b>0.798</b> | 0.707        | 0.870        | 0.775        |
| Large      | Student | 803K   | 784 KB    | <b>0.593</b> | <b>0.798</b> | <b>0.714</b> | <b>0.881</b> | <b>0.812</b> |
| XLarge     | Student | 1.62M  | 1.58 MB   | 0.551        | 0.794        | 0.690        | 0.830        | 0.726        |

Table 4: Committed healthy-to-impaired transfer-learning summaries. The CNN branch benefits consistently after finetuning; the LSTM branch improves on exact-match and finger accuracy but loses ground on ranking-sensitive metrics.

| Model / Stage | Exact-match  | Finger Acc.  | Macro F1     | Macro AUROC  | Macro AUPRC  |
|---------------|--------------|--------------|--------------|--------------|--------------|
| CNN pre.      | 0.465        | 0.760        | 0.664        | 0.833        | 0.699        |
| CNN fine.     | <b>0.585</b> | <b>0.800</b> | <b>0.680</b> | <b>0.863</b> | <b>0.767</b> |
| LSTM pre.     | 0.523        | 0.752        | <b>0.679</b> | <b>0.858</b> | <b>0.773</b> |
| LSTM fine.    | <b>0.574</b> | <b>0.777</b> | 0.660        | 0.830        | 0.736        |

from 0.764 to 0.812. Second, the compact Nano and Micro students remain close to the larger models despite much smaller INT8 footprints, which is the clearest current evidence for the project’s edge-oriented design goal. Third, the teacher results provide a meaningful internal reference. Among the committed teacher checkpoints, ResNet-152 is the strongest teacher with 0.588 exact-match accuracy and 0.698 macro F1, yet CNN-Large still slightly exceeds it on exact-match accuracy, macro F1, macro AUROC, and macro AUPRC. This does not prove that distillation is already responsible for the gain, because the repository does not yet commit a comparable distilled-student result set. It does show that the tuned student branch is already competitive with its larger teacher family.

### 5.3 Improvement paths for metrics and speed

Transfer-learning summaries in Table 4 show a deliberately asymmetric result: one path works clearly, one does not. For the CNN branch, healthy-to-impaired staging improves exact-match accuracy from 0.465 to 0.585 and finger accuracy from 0.760 to 0.800 while also improving macro AUROC and macro AUPRC. For the LSTM branch, finetuning improves exact-match and finger accuracy but lowers macro F1, macro AUROC, and macro AUPRC relative to the healthy pretraining summary. The practical conclusion is that healthy-arm pretraining is already a credible improvement path for the CNN family in this repository, but not a universally beneficial recipe across architectures.

Distillation is best interpreted differently. The repository now contains teacher-ensemble distillation code, explicit multilabel distillation losses, threshold-tuning utilities, and latency-benchmarking scripts, all of which are aligned with the project’s goal of improving the speed-quality trade-off for Raspberry Pi-class deployment. However, the current artifact set still lacks a mature distilled-student evaluation bundle and committed on-device timing logs. The distillation story is therefore

Table 5: Per-finger F1 score by model family. The optimized CNN-Large replaces the legacy direct CNN row for the finger-level comparison. Bold numbers mark the strongest model for each finger.

| Finger | LSTM         | GNN          | CNN-Large    |
|--------|--------------|--------------|--------------|
| Thumb  | 0.716        | <b>0.776</b> | 0.729        |
| Index  | 0.723        | 0.696        | <b>0.750</b> |
| Middle | <b>0.712</b> | 0.689        | 0.698        |
| Ring   | <b>0.696</b> | 0.688        | 0.695        |
| Little | 0.680        | 0.678        | <b>0.696</b> |

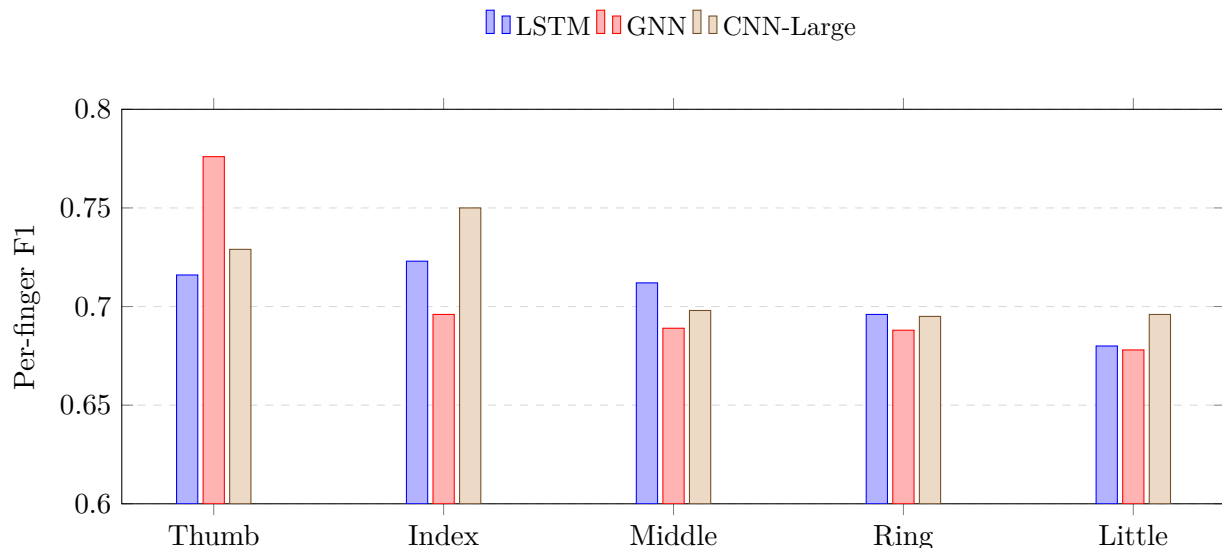


Figure 3: Per-finger F1 comparison derived from the committed metric files. Aggregate CNN gains coexist with output-specific strengths for the LSTM and GNN baselines.

architectural and procedural rather than empirical at this stage: it is an implemented path for future speed and compression gains, not yet a closed result in the same sense as the direct or transfer tables.

#### 5.4 Finger-level behavior and external positioning

Finger-wise scores show that the ranking is not uniform across all outputs. Table 5 compares the LSTM direct baseline, the GNN direct baseline, and the best optimized CNN student. The GNN remains strongest on the thumb label, the optimized CNN-Large leads on index and little finger, and the LSTM remains slightly strongest on middle and ring finger. This pattern suggests that aggregate CNN gains do not erase all output-specific variation; different inductive biases still matter at the finger level.

To show where this project stands relative to the literature, Table 1 should be read alongside the internal results rather than against them. Prior stroke-decoding papers often report much higher accuracies than our exact-match scores, but they typically solve easier multiclass tasks with fewer gestures, fewer labels, smaller cohorts, or healthy-data pretraining. Our current numbers are stricter in at least three ways: they use a five-finger multilabel formulation, they are derived from impaired-arm PHYSIOMIO recordings with patient-level splits, and the headline exact-match metric

requires all five finger labels to be correct simultaneously. For that reason, the more meaningful external takeaway is not that PSR-PIPELINE already leads the literature, but that it now occupies a technically credible position: stronger than a simple proof-of-concept, richer than a single-model benchmark, and explicit enough to support future work on transfer, distillation, and embedded deployment.

The results should nevertheless be read with an evidence boundary in mind. The project now includes code for teacher-ensemble distillation, per-finger threshold tuning, and CPU latency measurement, but it does not yet commit a mature result bundle produced by those new utilities, nor does it store on-device Raspberry Pi 5 timings. The current paper can therefore support a stronger research narrative than the previous draft, but not a full claim of embedded deployment readiness.

## 6 Discussion

The strongest conclusion supported by the updated evidence is that PSR-PIPELINE now supports a credible software research program for sEMG finger-intent decoding in post-stroke neurorehabilitation, not just a loose collection of model experiments. Another researcher or engineer can trace the workflow from raw PHYSIOMIO recordings through preprocessing, feature extraction, tensor adaptation, direct baselines, optimized CNN students, teacher references, transfer-learning summaries, and deployment-facing utilities without reconstructing the pipeline by hand.

The updated benchmark also changes the model-level interpretation. The earlier baseline story was genuinely three-way: the LSTM offered the best direct exact-match accuracy, the GNN offered the best baseline macro F1 and AUPRC, and the original CNN was not yet dominant. That baseline-first framing still matters because it shows the problem was not trivial and because it explains the rationale for studying multiple inductive biases under one pipeline. What changes in the updated paper is the second half of the story. Once the merged CNN student and teacher results are included, the strongest committed artifacts are compact convolutional models that outperform the older direct baselines on the headline aggregate metrics while remaining small enough to fit the intended embedded setting after quantization.

The transfer-learning summaries point toward a second useful lesson. Healthy-to-impaired staging appears substantially more promising for the CNN branch than for the LSTM branch. That asymmetry may reflect architecture-specific robustness, optimization differences, or the way the shared feature representation interacts with each model family. The project does not yet include the ablations needed to explain the mechanism, but it does provide enough evidence to motivate transfer learning as a real research direction rather than a speculative aside. This finding also aligns with the recent stroke-specific literature, which increasingly treats healthy-data pretraining as a practical way to compensate for the scarcity and heterogeneity of paretic sEMG [13].

The project’s deployment story is also more coherent than before, but it is still incomplete. The codebase now includes teacher-ensemble distillation, threshold calibration, and latency benchmarking, and the student model sizes are explicitly framed around Raspberry Pi 5-class constraints. Those are meaningful steps toward embedded inference. However, the paper still cannot claim a validated deployment path because the committed artifact set does not yet include end-to-end Raspberry Pi measurements, power or thermal observations, or a reproducible distilled-student benchmark pack aligned with the rest of the evaluation tree.

Several limitations remain and should be read plainly. First, the benchmark still centers on a single dataset and one main preprocessing configuration, with no committed ablation package isolating

the contribution of filtering, denoising, feature design, threshold choice, or graph connectivity. Second, the GNN implementation uses dense window graphs, which may scale poorly as sequence length grows. Third, the transfer-learning summaries are committed as high-level JSON rather than as full rerunnable experiment bundles with harmonized plotting and checkpoint metadata. Fourth, external literature comparisons remain only contextual because the field is fragmented across different populations, gesture vocabularies, electrode layouts, and label spaces. Fifth, the broader rehabilitation objective still lacks the evidence that matters most for translation: on-device timing, robust behavior across users and sessions, and human-subject evaluation of whether predicted intent meaningfully improves therapy.

Taken together, these limitations do not weaken the value of the current paper as a research manuscript for arXiv. They define the next layer of work. The clearest path forward is to push the same artifact discipline into distilled-student evaluation, Raspberry Pi 5 benchmarking, and broader cross-patient or cross-session validation so that the software paper can connect cleanly to a later deployment paper.

## 7 Conclusion

This paper presents PSR-PIPELINE as a software research framework for sEMG-based finger-intent prediction in post-stroke neurorehabilitation. The contribution is not a new standalone network, but a reproducible experimental program that makes ingestion, preprocessing, feature extraction, tensor adaptation, architecture comparison, transfer learning, and deployment-screening utilities explicit and reusable.

The evaluation now tells a clearer story. The project begins with three direct model families, LSTM, CNN, and GNN, whose baseline results show complementary strengths rather than a trivial winner. It then advances through optimized CNN students and internal ResNet teachers, where CNN-Large becomes the strongest committed artifact, and through transfer-learning experiments that help the CNN branch but not the LSTM branch consistently. Distillation, threshold tuning, and latency benchmarking now exist as concrete deployment-facing pathways, but they still need the same artifact completeness as the direct benchmark path. The main remaining gap is therefore not framing. It is evidence: distilled-student result packs, Raspberry Pi 5 timing validation, and broader robustness studies that can carry this software paper toward a fully supported deployment study later.

## References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [2] Alexey Anastasiev, Hideki Kadone, Aiki Marushima, Hiroki Watanabe, Alexander Zaboronok, Shinya Watanabe, Akira Matsumura, Kenji Suzuki, Yuji Matsumaru, and Eiichi Ishikawa. Supervised myoelectrical hand gesture recognition in post-acute stroke patients with upper limb paresis on affected and non-affected sides. *Sensors*, 22(22):8733, 2022. doi: 10.3390/s22228733.
- [3] Manfredo Atzori, Arjan Gijsberts, Claudio Castellini, Barbara Caputo, Anne-Gabrielle Mittaz Hager, Simone Elsig, Giorgio Giatsidis, Franco Bassetto, and Henning Müller. Electromyography

- data for non-invasive naturally-controlled robotic hand prostheses. *Scientific Data*, 1:140053, 2014. doi: 10.1038/sdata.2014.53.
- [4] Issam Boukhenoufa, Xiaojun Zhai, Victor Utti, Jo Jackson, and Klaus McDonald-Maier. Wearable sensors and machine learning in post-stroke rehabilitation assessment: A systematic review. *Biomedical Signal Processing and Control*, 71:103197, 2022. doi: 10.1016/j.bspc.2021.103197.
  - [5] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, 2017.
  - [6] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
  - [7] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
  - [8] Julian Ilg, Alexander C. R. Oldemeier, Marie Fieweger, Luca Deuschel, Peter Rieckmann, Peter Young, Sabine Krause, and Tim C. Lueth. Physiomio: Bilateral and longitudinal hd-semg dataset of 16 hand gestures from 48 stroke patients. *Scientific Data*, 13(1):19, 2026. doi: 10.1038/s41597-026-06557-0.
  - [9] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
  - [10] Sang Wook Lee, Kristin Wilson, Blair A. Lock, and Derek G. Kamper. Subject-specific myoelectric pattern classification of functional hand movements for stroke survivors. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 19(5):558–566, 2010. doi: 10.1109/TNSRE.2010.2079334.
  - [11] Jianfeng Li, Xinyu Jiang, Jiahao Fan, Yanjuan Geng, Fumin Jia, and Chenyun Dai. Deep end-to-end transfer learning for robust inter-subject and inter-day hand gesture recognition using surface emg. *Biomedical Signal Processing and Control*, 100:106892, 2025. doi: 10.1016/j.bspc.2024.106892.
  - [12] Wei Li, Ping Shi, and Hongliu Yu. Gesture recognition using surface electromyography and deep learning for prostheses hand: State-of-the-art, challenges, and future. *Frontiers in Neuroscience*, 15:621885, 2021. doi: 10.3389/fnins.2021.621885.
  - [13] M. Mohammadiazni, K. Huszar, S. Peters, and A. L. Trejos. Hand gesture intention detection using semg and transfer learning in stroke survivors. *IEEE Journal of Biomedical and Health Informatics*, 2026. doi: 10.1109/JBHI.2026.3693109. Early access, published May 13, 2026.
  - [14] Maria Munoz-Novoa, Morten B. Kristoffersen, Katharina S. Sunnerhagen, Autumn Naber, Margit Alt Murphy, and Max Ortiz-Catalan. Upper limb stroke rehabilitation using surface electromyography: A systematic review and meta-analysis. *Frontiers in Human Neuroscience*, 16:897870, 2022. doi: 10.3389/fnhum.2022.897870.
  - [15] Sike Ni, Mohammed A. A. Al-qaness, Ammar Hawbani, Dalal Al-Alimi, Mohamed E. Abd Elaziz, and Ahmed A. Ewees. A survey on hand gesture recognition based on surface electromyography: Fundamentals, methods, applications, challenges and future trends. *Applied Soft Computing*, 166:112235, 2024. doi: 10.1016/j.asoc.2024.112235.



- [16] Ninapro Project. The Non-Invasive Adaptive Prosthetics (Ninapro) Database. <https://ninapro.hevs.ch/>, 2026. Project site, accessed July 28, 2026.
- [17] Post-Stroke Rehabilitation Project. PSR-Pipeline. <https://github.com/post-stroke-rehab/psr-pipeline>, 2026. GitHub repository, accessed July 28, 2026.
- [18] Raspberry Pi Ltd. Introducing: Raspberry Pi 5! <https://www.raspberrypi.com/news/introducing-raspberry-pi-5/>, 2023. Official product announcement, accessed July 28, 2026.
- [19] Panyawut Sri-Iesaranusorn, Attawit Chaiyaroj, Chatchai Buekban, Songphon Dumnin, Ronachai Pongthornseri, Chusak Thanawattano, and Decho Surangsiriat. Classification of 41 hand and wrist movements via surface electromyogram using deep neural network. *Frontiers in Bioengineering and Biotechnology*, 9:548357, 2021. doi: 10.3389/fbioe.2021.548357.
- [20] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- [21] Peter D. Welch. The use of fast fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.
- [22] Mingde Zheng, Michael S. Crouch, and Michael S. Eggleston. Surface electromyography as a natural human-machine interface: A review. *arXiv preprint arXiv:2101.04658*, 2021.